

LLaGA 中文精读导读：把一个节点“翻译”成大模型听得懂的“外语词”

原文：Runjin Chen, Tong Zhao, Ajay Jaiswal, Neil Shah, Zhangyang Wang (ICML 2024)

本导读为面向初学者的中文学习笔记：含要点翻译、公式精讲与图表解读。

论文卡片

标题	LLaGA: Large Language and Graph Assistant (大语言与图助手)
作者	Runjin Chen, Tong Zhao, Ajay Jaiswal, Neil Shah, Zhangyang Wang
机构	德州大学奥斯汀分校 (UT Austin)、Snap Inc.
会议	ICML 2024 (国际机器学习大会)
arXiv	2402.08170
代码	https://github.com/VITA-Group/LLaGA
方向	图 (Graph) × 大模型 × 后训练 (投影器对齐 / 表示对齐)
一句话	把一个节点和它的邻居重排成“结构感知 token 序列”，用一个轻量投影器映射进 LLM 词向量空间当“外语词”，冻结 LLM、只训投影器，一个统一模型同时做节点分类 / 链接预测 / 节点描述，还能零样本迁移。

1 一、这篇论文在解决什么问题

现实世界里大量数据天生是“图”：社交网络、生物网络、推荐系统、引文网络……它们由**节点**（实体）和**边**（关系）组成。专门处理图的工具叫**图神经网络（Graph Neural Network, GNN）**，它靠“消息传递 + 聚合”把每个节点变成一个向量，在图任务上很强。

另一边，**大语言模型（Large Language Model, LLM，如 GPT-4、LLaMA）**最近大放异彩——**一个模型就能干很多种任务**，还会用自然语言解释自己的答案。于是一个自然的念头冒出来：能不能让 LLM 也来处理图？但作者指出，已有做法各有硬伤：

1. **GNN 不擅长多任务**：GNN 通常只为“某一个任务”训练，换个任务往往就要换“任务专用的脑袋（分类头）”重新调，做不到一个模型通吃。

2. **把图“说”成文字效果差**：有人尝试把图结构直接用自然语言描述（比如“节点 A 连着 B 和 C ……”）喂给 LLM。但这种描述又啰嗦又不直观，LLM 在基础图任务上反而表现很差。
3. **指令微调会“偏科”**：像 InstructGLM 那样为某个图任务专门微调 LLM，虽然该任务变好了，却牺牲了 LLM 的通用性，在别的任务上变弱。
4. **预训练图编码器未必合身**：像 GraphGPT 那样先用一个预训练好的图 Transformer 抽结构信息，但这个“现成编码器”未必能为每个下游任务蒸馏出恰好需要的结构特征。

本文要回答的核心问题：怎样设计一个框架，能跨多种任务、多种领域有效地把图的结构信息编码出来，让 LLM 看得懂，同时又不破坏 LLM 原本的通用能力？这就是 LLaGA。

2 二、摘要中译

图神经网络 (GNN) 推动了图结构数据分析的发展。近来，以 GPT-4 为代表的大语言模型 (LLM) 开启了深度学习的新纪元；然而，由于“把图结构翻译成语言”本身就很困难，将 LLM 应用到图数据上仍面临独特挑战。为此，我们提出**大语言与图助手 (Large Language and Graph Assistant, LLaGA)**，一个能有效整合 LLM 能力来处理复杂图结构数据的创新模型。LLaGA 在**保留 LLM 通用性**的同时，把图数据改造成与 LLM 输入兼容的格式。具体做法是：把图中的节点**重新组织成结构感知的序列**，再通过一个**通用投影器 (versatile projector)** 把它们映射进 token 词向量空间。LLaGA 在**通用性、泛化性与可解释性**三方面都很出色：它能在不同数据集和任务上始终表现良好，能把能力扩展到**未见过的数据集或任务**，还能**为图提供解释**。我们在主流图基准上的大量实验表明，LLaGA 用一个模型就在四个数据集、三种任务上取得出色性能，在监督与零样本两种场景下都超过了当前最强的图模型。代码见 <https://github.com/VITA-Group/LLaGA>。

3 三、三大核心贡献（论文称为三个特点）

1. **通用性 (Versatility)**：用一个简单却普适的方法编码图的结构细节，再用**单个通用投影器**把图空间和 token 空间对齐。这个投影器能跨多个数据集处理多种图任务，**不需要任务专用的改动**——而且效果甚至能超过那些为单一任务量身定做的专用图模型。
2. **泛化性 (Generalizability)**：因为图与 token 空间被全面对齐了，LLaGA 不仅在训练见过的数据/任务上强，还能**不做任何额外微调**就泛化到从没见过的数据集和任务（零样本）。
3. **可解释性 (Interpretability)**：LLaGA 能对节点嵌入给出**详细的文字解释**（节点描述任务），让人能看懂它“为什么这么判断”。

4 四、预备知识（小白先看这里）

【小白补丁】什么是“文本属性图” (text-attributed graph, TAG)

就是节点上带文字的图。比如引文网络里，每个节点是一篇论文，它的“文字属性”就是论文的标题 + 摘要；电商网络里每个节点是一件商品，文字属性是商品描述。本文主要针对这种图。有了文字，就能用语言模型把它编码成向量。

【小白补丁】什么是“token 嵌入空间”和“投影器 (projector)”

LLM 内部不直接读文字，而是先把每个词 (token) 查成一个向量，这些向量所在的空间就叫 **token 嵌入空间 (词向量空间)**。本文的“投影器”是一个很小的神经网络 (一个 MLP)，作用是把“图节点的向量”搬到这个词向量空间里，让它伪装成 LLM 眼中的一个“词”。这正是 LLaVA 等“多模态 LLM”把图像塞进 LLM 的同款套路，只不过这里塞的是“图”。

【小白补丁】什么是“冻结 LLM、只训投影器” (这就是“后训练”的体现)

传统微调要改 LLM 的全部参数，又慢又贵，还容易让它“忘掉”原有的语言本领。本文反其道而行：**把 LLM 整个冻住、一个参数都不改**，只训练那个小投影器。代价极小，却能把“图的结构信息”对齐进 LLM。

形式化记号 (看不懂可跳过，用到再回看)：一张图记作 $G = (V, E, X)$ ，其中 V 是节点集、 E 是边集、 X 是节点的属性信息。每个节点 $v_i \in V$ 带一个属性特征 $x_i \in X$ (本文里是**文字**)。另外用 N_v^k 表示节点 v 的**第 k 跳邻居集合** (即“走 k 步能到的邻居”)。用一个现成的文本编码模型 ϕ (如 SBERT、RoBERTa、SimTeG) 把节点文字编码成向量 $\phi(x_v)$ ——这一步把“文字”变成“能算相似度的数字向量”，是后面一切的基础。

5 五、数据集与任务：一个模型，三种任务，四个数据集

四个数据集 (详细统计见原文 Table 7)。引文网络里节点是论文、边是共同引用；电商网络里节点是商品、边是共同购买：

表 1: LLaGA 用到的四个数据集（数据源自原文 Table 7）。

数据集	领域	节点数	边数
Cora	引文网络 (citation)	2,708	5,429
Pubmed	引文网络 (citation)	19,717	44,338
ogbn-Arxiv	引文网络 (citation)	169,343	1,166,243
ogbn-Products	电商网络 (e-commerce)	2,449,029	61,859,140

可见数据集规模从几千节点 (Cora) 一路跨到两百多万节点 (Products)，稀疏程度也各不相同，能很好地考验方法。

三种任务（一个模型全包）：

- **节点分类 (node classification)**：判断一个节点属于哪一类（论文属于哪个研究领域、商品属于哪个品类）。评测指标是**准确率 (Accuracy)**。
- **链接预测 (link prediction)**：判断两个节点之间是否应该有一条边。指标也是准确率。
- **节点描述 (node description)**：让模型用一段话描述某个节点（“这个节点代表一篇关于……的论文”）。这是本文比较新颖的任务，用来把“节点嵌入”对齐到“具体的描述文字”，从而带来可解释性。指标是 **Sbert Score**（生成描述与真值描述的语义相似度）和 **Description Label Accuracy**（从描述里反推出的类别是否正确）。

【小白补丁】统一成“问答 (QA) 格式”是关键巧思

传统 GNN 做多任务，要给每个任务配一个“专用脑袋”和“专用损失函数”。LLaGA 不这样——它把所有任务都写成同一种“问答对话”格式：给一段提示词，里面塞入“节点序列”，让 LLM 像聊天一样把答案生成出来。于是不再需要任务专用的损失或分类头，一个统一框架就能通吃三种任务。

6 六、方法逐节精讲

LLaGA 的核心只有两步（见图 1）：**第一步**把“图结构”变成“节点序列”（这一步完全不含参数，纯靠规则重排）；**第二步**用投影器把“节点序列”变成“token 嵌入序列”塞进冻结的 LLM。整个训练里，**唯一要学的参数就是那个投影器**。

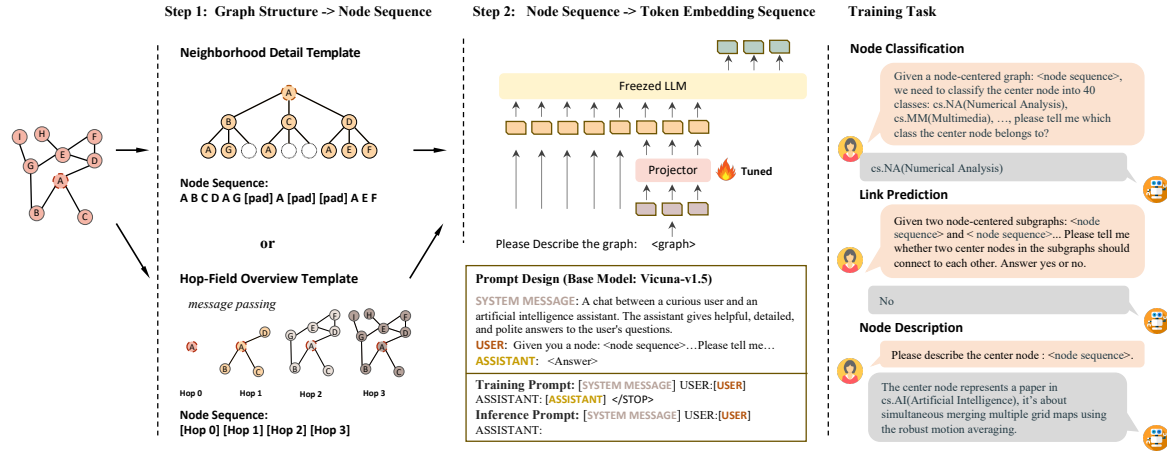


Figure 1. Illustration of LLaGA framework and its prompt design paradigm.

central node and its immediate surroundings. The second, the **Hop-Field Overview Template**, offers a summarized view of a node’s neighborhood, extendable to larger fields.

Neighborhood Detail Template is designed to elaborate on the detailed information of a node and its surrounding

to a relative structural position within the original graph.

Post conversion of the center node and its structural information into a node sequence, we shift to mapping them into the node embedding space. In the context of text-attributed graphs, we can utilize various off-the-shelf text encoding

图 1: LLaGA 总体框架与提示词设计范式 (原文 Figure 1, 图内英文为原文保留)。**Step 1:** 把中心节点周围的图结构, 用两种模板之一重排成定长的“节点序列”——上方 *Neighborhood Detail Template* (邻域细节模板) 把邻居展开成一棵计算树再按层遍历; 下方 *Hop-Field Overview Template* (跳数视野概览模板) 把每一跳的邻居汇总成一个向量。**Step 2:** 把节点序列经投影器 (Projector, 唯一可训练, 标 “Tuned”) 映射进 token 空间, 连同提示词一起喂给冻结的 LLM (Freezed LLM, 基座为 Vicuna-v1.5)。**右侧:** 三种训练任务统一成问答格式——节点分类、链接预测、节点描述。

6.1 第 1 步: 结构感知的“图 → 节点序列”翻译 (无参数)

图分析的基本单元是节点。作者设计了两种节点级模板, 从两个不同视角把“中心节点 + 它的邻域结构”摊平成一条定长序列。注意: 这一步不含任何可学习参数, 只是按规则搬运, 所以原始结构信息被完整保留、无需“蒸馏”。

模板一: 邻域细节模板 (Neighborhood Detail Template)——看得细。给定中心节点 v , 先以它为根建一棵形状固定的采样计算树。为每一跳设一个采样数 $n_1, n_2, \dots$ (n_i 是第 i 跳要采几个邻居)。从 v 的 1 跳邻居 N_v^1 里随机选 n_1 个; 如果邻居不够 n_1 个, 就用占位节点 [pad] 补齐到 n_1 个。这样每个邻居集合大小都恒为 n_1 。再对每个被选中的邻居, 递归地各采 n_2 个邻居当它的孩子, 不够同样用 [pad] 补。占位节点的孩子只能是占位节点。最后对这棵树做层序遍历 (按层从左到右), 就得到一条定长序列。

【小白补丁】 用图 1 左上角的例子走一遍

以 A 为根、两跳、每跳采 3 个。A 的一阶邻居是 {B, C, D}, 放在第二层。B 只有邻居 {A, G}, 补

成 $\{A, G, [\text{pad}]\}$; C、D 同理。最终得到一棵“3 叉满树”，层序遍历后序列是：A B C D A G [pad] A [pad] [pad] A E F。每个位置都唯一对应原图里的一个相对结构位置——这正是“结构感知”的含义。

把序列里每个节点用文本编码器 ϕ 编码成向量后，作者还在每个位置**额外拼接一个拉普拉斯位置嵌入 (Laplacian Embedding) U_i** ，进一步强化结构信息。记计算树的邻接矩阵为 A_{tree} ，其拉普拉斯矩阵做特征分解：

$$L = I - D^{-\frac{1}{2}} A_{tree} D^{-\frac{1}{2}} = U^T \Lambda U \quad (1)$$

其中 D 是 A_{tree} 的度矩阵， U 就是这棵模板树的拉普拉斯（位置）嵌入。

【小白补丁】为什么这个嵌入“只算一次”就够了

因为采样数固定，所有节点用的计算树形状都一模一样，所以 U 对所有图、所有节点都通用，只需在一开始算一次。它的作用类似 Transformer 里的“位置编码”——告诉模型“序列里这个位置在树里处于什么结构角色”。

于是第 i 个位置的最终节点嵌入 h_{v_i} 为（ $\parallel$ 表示向量拼接）：

$$h_{v_i} = \begin{cases} \mathbf{0} \parallel U_i, & \text{若 } v_i = [\text{pad}] \text{ (占位节点用全零向量)}; \\ \phi(x_{v_i}) \parallel U_i, & \text{否则 (真实节点用其文本编码)}. \end{cases} \quad (2)$$

最终，中心节点与其结构信息被转成定长的节点嵌入序列 $h_{v_1}, h_{v_2}, \dots, h_{v_n}$ 。

模板二：跳数视野概览模板 (Hop-Field Overview Template)——看得广。它不为每个邻居单独留一个向量，而是把每一“跳”的所有邻居汇总成一个向量。对中心节点 v ，第 i 跳的嵌入 h_v^i 通过**无参数的消息传递**（就是对上一跳邻居向量求平均）得到：

$$h_v^i = \frac{1}{|N_v^1|} \sum_{v' \in N_v^1} h_{v'}^{i-1}, \quad \text{其中 } h_v^0 = \phi(x_v). \quad (3)$$

这里 h_v^0 是节点自身的文本编码（第 0 跳）；逐跳平均后， h_v^i 就**浓缩了第 i 跳邻域的整体信息**。于是一串 $h_v^0, h_v^1, h_v^2, \dots$ （每跳一个向量）就代表了中心节点和它的多跳结构。

【小白补丁】两个模板怎么选？

细节模板保留每个邻居，适合“需要看清具体某个邻居”的任务（比如判断商品是不是“任天堂 Switch”那一类）；**概览模板**用一个向量概括一整跳，视野更大但牺牲了细节，适合“看大局”的任务（比如看一篇论文周围大体上都是哪些领域）。论文里把用细节模板的版本记作 **LLaGA-ND**，用概览模板的记作 **LLaGA-HO**。

6.2 第 2 步：用投影器把“节点序列”对齐进 token 空间

要让 LLM 看懂图输入，必须把节点嵌入空间和 LLM 的输入 token 空间对齐。做法是：用一个专门校准的投影器 f_θ （实现上就是一个简单的 MLP），把每个节点嵌入 h_i 映射成一个 token 嵌入 e_i ：

$$e_i = f_\theta(h_i). \quad (4)$$

于是节点嵌入序列 $h_1, h_2, \dots, h_n$ 就被转成对应的 token 嵌入序列 $e_1, e_2, \dots, e_n$ 。**划重点：投影器的参数 θ 是整个训练过程中唯一被更新的参数。**

6.3 第 3 步：对齐微调（Alignment Tuning）——训练目标

作者用前面那三种任务（节点分类、链接预测、节点描述）一起来训练这个投影器。把这三种任务混在一起训，能让投影器从多个角度理解图，练成一个“图嵌入 $\leftrightarrow$ token 嵌入”的通用翻译器，并顺带获得生成节点解释的能力。

训练时所有数据都写成聊天问答格式（基座 LLM 用 Vicuna，按其规范设好系统提示）。处理输入时，把提示词里的普通文字正常 tokenize；唯独遇到占位符 `<node sequence>` 时，就用投影后的节点嵌入 $e_1, \dots, e_n$ 原位替换它。训练目标很直接——**最大化生成正确答案的概率**：

$$\underset{\theta}{\text{maximize}} \quad p(X_{\text{answer}} | X_{\text{graph}}, X_{\text{question}}, X_{\text{system}}). \quad (5)$$

其中 X_{graph} 是（投影后的）图序列、 X_{question} 是问题、 X_{system} 是系统提示、 X_{answer} 是标准答案。注意优化变量只有 θ （投影器），LLM 本体全程冻结。

【小白补丁】一句话抓住整套方法

LLaGA = 把节点邻域规则化重排成“结构感知序列”（无参数，含位置嵌入） $\rightarrow$ 经一个小投影器变成 LLM 词向量空间里的“外语词” $\rightarrow$ 连同问题一起喂给冻结的 LLM 生成答案。全程只训投影器，所以又轻、又快、又不破坏 LLM 的通用能力。

7 七、实验怎么做、结果怎么读

实验设置：基座 LLM 默认用 Vicuna-7B-v1.5-16K；文本编码默认用 SimTeG；学习率 $2e-5$ 、批大小 16、只训 1 个 epoch（最小的数据集 Cora 复制 3 倍以补数据量）。细节模板采样 2 跳、每跳 10 个邻居；概览模板用 4 跳嵌入。对比的基线分三类：GNN（GCN、GraphSAGE、GAT、SGC、SAGN）、图 Transformer（NodeFormer）、以及通用大模型 GPT-3.5；还和同期工作 GraphGPT 比。

四种训练设定（一个比一个“通才”，越靠后越难，越能体现 LLaGA 价值）：

- **Single Focus：**单数据集、单任务（最专精）。

- **Task Expert**: 跨所有数据集、但只做单一任务。
- **Classification Expert**: 跨所有数据集，同时做节点分类和链接预测两项。
- **General Model**: 跨所有数据集，同时做**三项任务**（含节点描述）——最“全才”。

基线 GNN 做多任务时，靠“共享主干 + 多个任务专用分类头”；而 LLaGA 始终只用一个**投影器**通吃。

主结果（节选自原文 Table 1）。下表挑了有代表性的几行：在最简单的 Single Focus 设定里，LLaGA 已能与最强 GNN 平起平坐甚至更好；越到“多任务”设定，差距越明显。

表 2: 节点分类与链接预测准确率 (%) 对比（节选自原文 Table 1；越高越好，加粗为该设定下最优）。

设定	模型	Arxiv	Products	Pubmed	Cora
Single Focus（分类）	GraphSAGE	76.29	82.87	94.87	88.89
	LLaGA-HO-7B	76.66	84.67	95.03	89.22
Classification Expert（分类）	GCN	70.95	80.02	89.00	82.77
	LLaGA-HO-7B	75.99	83.32	94.80	89.30
Classification Expert（链接预测）	GCN	87.69	92.88	72.28	78.35
	LLaGA-HO-7B	94.30	96.05	88.64	88.53

怎么读这张表：作者总结出三点观察。**观察 1**——LLaGA 在所有设定下都优于基线，尤其在多任务场景。**观察 2**——很多基线一进多任务场景就**大幅掉分**（比如上表里 GCN 在 Classification Expert 的 Pubmed 链接预测只有 72.28），而 LLaGA **几乎不掉甚至还升**，说明它擅长跨数据/任务提取共性。**观察 3**——两个模板各有所长：细节模板擅长需要具体邻居信息的任务，概览模板擅长需要大视野的任务。

对比同期工作 GraphGPT（源自原文 Table 2）：LLaGA 最通用的模型一口气训了 12 个任务，而 GraphGPT 最通用的模型只训了 3 个任务；即便如此，LLaGA 在这三项上仍**全面胜出**（例如 Pubmed 链接预测 89.81 对 GraphGPT 的 80.26）。

节点描述与可解释性（RQ2，源自原文 Table 4）：LLaGA 生成的描述与真值的 Sbert 相似度**远高于**“两条随机样本”的 Base 基准值（例如 Pubmed 上 0.69 对 0.49），且从描述里反推类别的准确率也很高。原文 Table 3 还展示了实例：即便偶尔预测的标签和真值不同，它给出的解释也**合情合理**（比如把一篇抑郁症检测论文判成“社交与信息网络”而非“机器学习”，但理由站得住脚）。

零样本迁移（RQ3，源自原文 Table 5 / Table 8）：训练时没见过的数据集，测试时直接上。这里 LLaGA 的优势最抢眼：

表 3: 链接预测的零样本迁移准确率 (%) (数值源自原文 Table 5; 越高越好)。

训练 → 测试	模型	准确率
Arxiv+Pubmed → Cora (同领域)	GraphGPT-7B	50.74
	LLaGA-HO-7B	87.35
Arxiv+Pubmed+Cora → Products (跨领域)	GraphGPT-7B	50.74
	LLaGA-HO-7B	92.99

注意第二行是**跨领域**迁移 (训练全是引文网络, 测试却是电商网络), LLaGA 仍达到 92.99%, 说明它学到的“图 ↔ token 对齐”能**迁移到差异很大的新领域**。节点分类的零样本同样大幅领先 GraphGPT (原文 Table 8, 例如 Arxiv+Pubmed→Cora 在“节点嵌入 + 文本属性”提示下达到 59.59%, GraphGPT 仅 44.65%)。

模板消融 (RQ4, 源自原文 Table 6): 去掉模板、只用中心节点自己的嵌入做预测, 性能明显下降——尤其在**高度依赖结构信息**的链接预测上掉得最多 (例如 Pubmed 从约 92.36 掉到 78.19)。这证明**两个结构模板确实把邻域结构信息有效编码进去了**。

鲁棒性补充 (附录 C / D): 换文本编码器 (SBERT、RoBERTa) 或换基座 LLM (LLaMA2-7B、甚至更轻的 OPT-2.7B), LLaGA 都**稳定优于 GNN 基线**, 说明方法对“编码器/基座”的选择不挑食。

8 八、附录里值得一看的补充

- **数据划分** (附录 A): 节点级任务用标准 train/val/test 划分 (Arxiv 6:2:3、Products 8:2:90、Pubmed 与 Cora 均 6:2:2); 链接预测的训练/测试节点对分别从对应集合里采样。
- **节点分类零样本更难** (附录 B): 因为不同任务标签集不同, 比链接预测更具挑战; 作者借助“节点描述任务”先建立图与语义空间的通用对齐, 再迁移到分类, 效果仍超 GraphGPT。
- **换文本编码器** (附录 C, Table 9): SBERT / RoBERTa 都能用, LLaGA 始终领先 GNN。
- **换基座 LLM** (附录 D, Table 10): Vicuna-7B / LLaMA2-7B / OPT-2.7B 都行, 连 2.7B 的小模型也有效。
- **方差信息** (附录 E, Table 11): 在 Cora、Pubmed 上重复 5 次, 结果方差很小 (如 Cora 分类 88.86 ± 0.78), 说明稳定。

9 九、小白知识点卡片 (速查)

- **文本属性图 / TAG**: 节点带文字的图 (引文网络、电商网络等)。

- **GNN (图神经网络)**: 靠“消息传递 + 聚合”把节点编码成向量的网络; 强但不擅长多任务。
- **token 嵌入空间 / 词向量空间**: LLM 把每个词查成向量后所在的空间。
- **投影器 (projector)**: 一个小 MLP, 把图节点向量“搬进”词向量空间当“外语词”; 本文唯一可训练部件。
- **结构感知序列**: 把中心节点的邻域按模板规则摊平成定长序列, 每个位置对应一个固定的结构角色。
- **邻域细节模板 / 跳数视野概览模板**: 两种重排方式, 一个“看得细”、一个“看得广”(对应 LLaGA-ND / LLaGA-HO)。
- **拉普拉斯位置嵌入**: 给序列每个位置加的“结构位置编码”, 因模板形状固定故只算一次。
- **占位节点 [pad]**: 邻居不够时补的空节点, 用全零向量表示。
- **冻结 LLM (frozen LLM)**: 训练时完全不改 LLM 参数, 保住其通用能力。
- **零样本 (zero-shot)**: 在训练时没见过的数据集/任务上直接评测, 考验迁移能力。
- **Vicuna / SimTeG / SBERT**: 分别是本文用的基座 LLM、默认文本编码器、可选文本编码器。

10 十、一句话总结与定位

一句话: LLaGA 把“一个节点 + 它的邻域结构”用无参数的模板重排成“结构感知 token 序列”, 再用一个小投影器把它映射进 LLM 的词向量空间当“外语词”, 冻结整个 LLM、只训这个投影器, 就让一个统一模型同时做好节点分类、链接预测、节点描述, 还能零样本迁移到全新的数据集和领域。

在“图 × 后训练”谱系里的位置: LLaGA 是“表示对齐 / 投影器路线”的代表作——它不改 LLM、不靠预训练图编码器、也不把图“说成”囫圇的自然语言, 而是把图直接编码成序列后投影对齐。与之相对: 同系列里 G-Retriever 走“图上检索 (GraphRAG) + 软提示”、InstructGraph/GraphWiz 走“指令微调 + DPO”、G1 走“强化学习”。你可以对照着看它们各自把“图的信息”塞进 LLM 的方式有何不同——LLaGA 的特色就是“轻投影器对齐 + 冻结基座 + 一模型多任务 + 强零样本”。

动手复现: 官方代码 <https://github.com/VITA-Group/LLaGA> (含两种模板的构造、投影器训练、以及 Vicuna 基座的统一问答训练流程), 是入门“图 × LLM 表示对齐”很好的练手项目。